Programación Orientada a Objetos (POO) en C++: Simulador de Combate Espacial en Consola (class Nave)

Informe Técnico de Laboratorio de Programación

Autores: Sofia Mojica, Gabriel Zacarías, Mateo Roberts y Leonel De Armas
Institución: E.E.S.T. N.° 1 “Eduardo Ader” Vicente López
Materia: Laboratorio de Programación (LPR), 5.° año 3.° A-B
Docentes a cargo: Prof. York Mansilla y Prof. Yamil Ganduglia
Fecha: Octubre de 2026

Introducción

El presente informe técnico documenta el desarrollo y la implementación de la **Actividad 8**, correspondiente al segundo cuatrimestre de la materia Laboratorio de Programación (LPR). El propósito central de esta práctica consiste en dar el salto evolutivo desde las estructuras de datos tradicionales (`struct`) hacia la Programación Orientada a Objetos (POO) utilizando el lenguaje C++ (Stroustrup, 2013). Para ello, se desarrolló un motor de juego de naves espaciales en consola, aplicando conceptos fundamentales de la ingeniería de software como el encapsulamiento, el diseño modular mediante clases (`class`), constructores, métodos y el manejo de objetos en memoria.

Objetivos

Objetivo general

- Aplicar los principios de la Programación Orientada a Objetos en C++ mediante el desarrollo de un simulador de combate espacial en consola basado en la clase `Nave`.

Objetivos específicos

- Diferenciar `struct` y `class` según la accesibilidad predeterminada de sus miembros.
- Aplicar el encapsulamiento para proteger el estado interno de la nave.
- Inicializar los objetos de la clase mediante un constructor.
- Incorporar una funcionalidad nueva (puntuación acumulativa) e integrarla en la clase, el bucle principal y el HUD.
- Documentar el trabajo con control de versiones y formato APA v7.

Marco Teórico

De struct a class

En C++, `struct` y `class` permiten definir tipos de datos propios que agrupan atributos y funciones miembro. La diferencia técnica fundamental radica en la accesibilidad predeterminada: los miembros de un `struct` son públicos por defecto, mientras que los de una `class` son privados (Stroustrup, 2013). La Programación Orientada a Objetos aprovecha esta posibilidad para combinar datos y comportamiento en una misma unidad, el objeto, y para controlar quién puede acceder a su estado interno.

Encapsulamiento y modificadores de acceso

El encapsulamiento consiste en ocultar el estado interno de un objeto y permitir su modificación solo a través de una interfaz pública de métodos. Los modificadores de acceso `private` y `public` delimitan qué miembros pueden utilizarse desde fuera de la clase (cplusplus.com, s. f.). De este modo, la clase puede validar cada cambio y mantener sus datos siempre en un estado coherente (Stroustrup, 2013).

Constructores

Un constructor es una función miembro especial que lleva el mismo nombre que la clase, no declara tipo de retorno y se ejecuta automáticamente cuando se crea un objeto (cplusplus.com, s. f.). Su función es inicializar los atributos con valores válidos desde el primer momento, de modo que el objeto nunca exista en un estado indefinido (Stroustrup, 2013).

Desarrollo

Descripción del simulador

El simulador es un motor de juego de naves espaciales en consola. La clase `Nave` modela la nave del jugador con sus coordenadas de posición (x, y) y su cantidad de vidas. El bucle principal, controlado por la condición `while (!gameOver)`, mantiene el juego en ejecución, detecta las colisiones entre balas y asteroides y actualiza el HUD (interfaz de información) en consola.

Diseño de la clase Nave

La Figura 1 muestra un fragmento ilustrativo de la clase con sus atributos privados, su constructor y sus métodos públicos. Al declarar `vidas` como atributo privado, una asignación directa desde `main()` como `jugador.vidas = -999;` no compila: g++ informa el error: *'int Nave::vidas' is private within this context*. La única vía para modificar el estado es a través de métodos como `reducirVida()`, que validan la operación.

Figura 1: Fragmento Ilustrativo de la Clase Nave

```
cpp
class Nave {
private:
   int x, y;     // posición en la consola
```

```
    int vidas;   // vidas restantes
    int score;   // puntaje acumulado

public:
    // Constructor: inicializa el objeto con valores válidos
    Nave(int _x, int _y, int _vidas) : x(_x), y(_y), vidas(_vidas), score(0) {}

    // Métodos controlados: validan antes de modificar el estado
    void reducirVida() { if (vidas > 0) vidas--; }
    void incrementarScore() { score++; }

    // Acceso de solo lectura al estado interno
    int getVidas() const { return vidas; }
    int getScore() const { return score; }
    bool estaViva() const { return vidas > 0; }
};
```

Usa el código con precaución.
Nota: Fragmento elaborado a partir de los atributos y métodos descritos en este informe; no reproduce el código completo del simulador.

Cuestionario Técnico de Control

struct vs class

¿Cuál es la principal diferencia técnica entre un struct (visto en Act. 7) y una clase, especialmente respecto a la sección private y public?
La diferencia técnica fundamental entre un `struct` y una `class` en C++ radica en la accesibilidad predeterminada de sus miembros (Stroustrup, 2013):

- **struct:** Todos los miembros son públicos (`public`) por defecto, lo que significa que cualquier parte del código puede acceder a ellos y modificarlos libremente.
- **class:** Los miembros son privados (`private`) por defecto. Esto obliga al programador a definir explícitamente las secciones de visibilidad, protegiendo los datos internos y estableciendo una interfaz controlada mediante métodos públicos.

Encapsulamiento

¿Por qué es peligroso dejar la variable salud como pública? ¿Qué desastre podría causar un programador novato desde el main()?
Dejar variables críticas como la salud o la energía de una nave como públicas elimina las barreras de seguridad del sistema. Si un programador novato accede directamente desde la función `main()`, podría asignar valores ilógicos o inválidos (por ejemplo, `jugador.vidas = -999` o `jugador.salud = 500000`). Esta asignación directa corrompe el estado interno del objeto, rompe la lógica del videojuego y provoca comportamientos erráticos o cierres inesperados (bugs difíciles de depurar). El encapsulamiento previene esto al obligar

a modificar el estado exclusivamente a través de métodos controlados (como `reducirVida()`) que validan las reglas del programa (Stroustrup, 2013).

Constructor

Explicá con tus palabras para qué sirve el bloque Nave(string _nombre, ...) dentro del código.

El constructor (`Nave(int x, int y, int _vidas)`) es un método especial que se ejecuta automáticamente en el preciso instante en que se crea o instancia un objeto de la clase `Nave` (cplusplus.com, s. f.). Su función principal es inicializar los atributos internos del objeto (las coordenadas de posición x, y y la cantidad inicial de vidas) con valores válidos y coherentes, evitando que las variables queden con “basura” en la memoria RAM y garantizando que la nave nazca lista para operar en el juego.

Desafío de Funcionalidad

Explicá con tus palabras qué funcionalidad elegiste agregar de las 15 opciones y en qué partes del código tuviste que insertarla para que funcione.

Para superar el desafío obligatorio, se implementó un **sistema de puntuación acumulativa y visualización de score en el HUD**. Para integrarla, se realizaron las siguientes modificaciones:

1. **Clase Nave:** Se añadió un atributo privado `int score` y se inicializó en 0 dentro del constructor.
2. **Interfaz:** Se creó un método público `incrementarScore()` para modificar de forma segura el puntaje.
3. **Bucle principal:** Dentro del ciclo `while (!gameOver)`, se actualizó la detección de colisiones para llamar a `incrementarScore()` cada vez que una bala impacta contra un asteroide.
4. **HUD:** Se actualizó la rutina de renderizado de la interfaz gráfica en consola para reflejar en tiempo real los puntos obtenidos.

Distribución de Tareas y Roles

Integrante	Rol y contribuciones
Sofia Mojica	Redacción de la introducción, conceptualización teórica inicial sobre el salto de `struct` a `class` y estructuración general del documento bajo formato APA v7.
Gabriel Zacarías	Desarrollo de las respuestas del cuestionario técnico, profundizando en la explicación teórica del encapsulamiento, modificadores de acceso (`private`/`public`) y constructores.

Mateo Roberts	Implementación y documentación de la sección del Desafío de Funcionalidad, detallando la lógica de programación integrada en C++ y las modificaciones del bucle principal.
Leonel De Armas	Administración de la configuración del repositorio *dual-remote* (GitHub), archivo `.gitignore`, licencia MIT, control de versiones y recopilación de capturas de ejecución.

Conclusiones

La transición de la programación estructurada basada en `struct` hacia la Programación Orientada a Objetos con `class` representa un avance indispensable en la formación técnica. A través de este simulador de combate espacial en consola, se comprobó empíricamente la utilidad del encapsulamiento para proteger la integridad de los datos y la modularidad para estructurar código limpio, mantenible y escalable. Asimismo, el uso de herramientas de control de versiones (Chacon y Straub, 2014) y la documentación formal bajo estándares de la industria reafirman el perfil profesional que exige el desarrollo de software moderno.

Referencias

- Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress.
- cplusplus.com. (s. f.). *Annyomous / Tutorials / C++ Language / Classes*.
- Stroustrup, B. (2013). *The C++ Programming Language* (4.ª ed.). Addison-Wesley.

Link al repositorio:

https://github.com/matuteXEN6/juegodenaves.git